

UNIT 3

1) Write short note on Network Address Translation.

Network Address Translation (NAT)

Definition:

Network Address Translation (NAT) is a technique used in computer networking to map private (internal) IP addresses to a public (external) IP address, enabling devices within a private network to communicate with external networks like the Internet.

Purpose:

1. **IP Address Conservation:** NAT allows multiple devices on a private network to share a single public IP address, helping to conserve IPv4 addresses.
2. **Security:** By hiding internal IP addresses, NAT adds a layer of security, as external users cannot directly access internal devices.
3. **Network Flexibility:** Internal networks can use any private IP address scheme without conflicting with public IP addresses.

How it Works:

- A NAT-enabled router modifies the source IP address of outgoing packets from a private IP to its own public IP.
- When a response comes back, NAT translates the destination IP from the public address back to the correct private IP, delivering the packet to the intended internal device.

Types of NAT:

1. **Static NAT:** One-to-one mapping between a private and a public IP address.
2. **Dynamic NAT:** Maps private IPs to a pool of public IPs on a first-come-first-serve basis.
3. **Port Address Translation (PAT) / NAT Overload:** Multiple private IPs share a single public IP using different port numbers.

Example:

- Private IP: 192.168.1.10 → Public IP: 203.0.113.5
- Internal device sends a request to the internet → NAT changes source IP to 203.0.113.5
- Response returns to 203.0.113.5 → NAT forwards it to 192.168.1.10

2) Explain the concept of Class Full and Class less Addressing

Classful Addressing

Saved memory full ⓘ

Definition:

Classful addressing is an **old IP addressing method** in which the **IP address space is divided into fixed classes (A, B, C, D, and E)** based on predefined **network and host portions**.

Key Points:

- IP address = **Network ID + Host ID**
- The division is determined by **fixed boundary bits**.
- No concept of subnet mask was used; the class itself defined how many bits were for the network and how many for hosts.

Class	Starting Bits	Range of First Octet	Default Subnet Mask	No. of Networks	Hosts per Network
A	0	1 – 126	255.0.0.0	128	~ 16 million
B	10	128 – 191	255.255.0.0	16,384	~ 65,000
C	110	192 – 223	255.255.255.0	2 million	254
D	1110	224 – 239	—	Multicasting	—
E	1111	240 – 255	—	Experimental	—

Example:

192.168.1.10 → Class C (default mask: 255.255.255.0)

Limitations:

- Wastage of IP addresses (fixed network size).
- No flexibility in dividing networks.

Classless Addressing (CIDR – Classless Inter-Domain Routing)

Definition:

Classless addressing **removes the fixed class boundaries** and allows the network and host portions to be defined **dynamically** using a **subnet mask** (written as `/n`).

Example:

192.168.1.0/26 → means first **26 bits** are network bits, remaining **6 bits** are for hosts.

Advantages:

- Efficient use of IP addresses (no wastage).
- Allows subnetting and supernetting.
- Improves routing efficiency.

Example:

In classless addressing, you can create subnets like:

192.168.1.0/26 → 64 IPs

192.168.1.64/26 → 64 IPs

instead of being forced to use one large Class C block of 254 IPs.

In Short:

Feature	Classful Addressing	Classless Addressing (CIDR)
Based on	Fixed Classes (A–E)	Variable Length Prefix
Subnet Mask	Default, based on class	User-defined (/n notation)
Flexibility	Rigid	Flexible
IP Utilization	Often wasteful	Efficient
Example	192.168.1.1 (Class C)	192.168.1.1/26

3) Compare Static and Dynamic Routing

1. Static Routing

Definition:

Static routing means the **routes are manually configured** by the network administrator. The router uses **fixed paths** to send data packets.

Features:

- Routes **do not change automatically** even if the network topology changes.
- Suitable for **small networks** with few routers.
- Requires **manual updates** when network changes occur.

Advantages:

- Simple to configure for small networks.
- No extra CPU or bandwidth usage for route calculations.
- More **secure** (no automatic updates from outside sources).

Disadvantages:

- Not scalable for large networks.
- Manual reconfiguration is needed if any link fails.
- Difficult to manage in complex topologies.

2. Dynamic Routing

Definition:

Dynamic routing uses **routing protocols** (like RIP, OSPF, or EIGRP) to **automatically learn and update** routes based on network changes.

Features:

- Routes are **automatically updated** when the network topology changes.
- Suitable for **large and complex networks**.
- Uses algorithms to determine the best path.

Advantages:

- **Automatic adaptation** to network changes or failures.
- **Scalable** and easier to manage in large networks.
- Provides **redundancy** and **fault tolerance**.

Disadvantages:

- Requires more CPU, memory, and bandwidth.
- More complex to configure initially.
- Slightly less secure if not properly managed.

3. Difference Table

Aspect	Static Routing	Dynamic Routing
Configuration	Manual	Automatic
Adaptability	Does not adapt to changes	Adapts automatically
Protocols Used	None	RIP, OSPF, EIGRP, BGP, etc.
Overhead	Low	High (due to updates and calculations)
Scalability	Poor	Excellent
Security	More secure	Comparatively less secure
Best Suited For	Small networks	Large networks
Example	Manually adding route using <code>ip route</code> command	Router learns routes using RIP or OSPF

4) Describe in short, the importance and working of ARP Protocol, what is ARP cache.

ARP (Address Resolution Protocol)

1. Importance:

- ARP is used to **find the physical (MAC) address** of a device from its **IP address** within a local network.
 - It enables communication between devices in the same LAN by mapping **IP addresses to MAC addresses**.
-

2. Working of ARP:

1. When a device wants to send data, it checks its **ARP cache** to see if it already knows the MAC address of the target IP.
 2. If not found, it **broadcasts an ARP Request** asking, "Who has IP x.x.x.x?"
 3. The device with that IP replies with an **ARP Reply** containing its MAC address.
 4. The sender then stores this IP-MAC mapping in its **ARP cache** for future use.
-

3. ARP Cache:

- The **ARP cache** is a **temporary table** stored in a device (like a router or computer) that keeps **recently resolved IP-to-MAC address mappings**.
 - It helps to **speed up communication** by avoiding repeated ARP requests for the same IP address.
-

✅ In short:

ARP translates IP addresses to MAC addresses for local communication. The ARP cache stores these mappings temporarily to make the process faster.

5) Differentiate between circuit switching, Packet switching, message Switching.

1. Circuit Switching

Definition:

A **dedicated communication path (circuit)** is established between sender and receiver for the entire duration of the communication.

Key Points:

- The path remains reserved until communication ends.
- Commonly used in **telephone networks**.
- Data is transmitted **continuously** and in order.

Advantages:

- Reliable and ordered delivery.
- Constant transmission rate (no delay once circuit is set).

Disadvantages:

- Wastes bandwidth when the line is idle.
- Setup time is required before transmission.

Example:

Traditional landline phone call.

2. Packet Switching

Definition:

Data is divided into **small packets**, and each packet is sent **independently** through the network using the best available path.

Key Points:

- No dedicated path; packets may take **different routes**.
- Each packet has a **header** with destination information.
- Used in **computer networks and the Internet**.

Advantages:

- Efficient use of bandwidth.
- Fault tolerant — if one path fails, packets can take another route.

Disadvantages:

- Packets may arrive **out of order** or delayed.
- Requires reassembly at the receiver's end.

Example:

Internet data transfer (emails, web browsing, etc.).

3. Message Switching

Definition:

Entire **messages are treated as single units** and stored at intermediate nodes before being forwarded ("store and forward" technique).

Key Points:

- No dedicated path.
- Each message is temporarily stored until the next link is free.
- Requires more storage at intermediate nodes.

Advantages:

- No need for dedicated path.
- Useful for **non-real-time** applications (like telegraphy).

Disadvantages:

- High delay due to store-and-forward process.
- Not suitable for real-time communication.

Example:

Older telegraph and email systems.

4. Difference Table

Feature	Circuit Switching	Packet Switching	Message Switching
Path	Dedicated path	Dynamic paths	No fixed path
Data Unit	Continuous data stream	Packets	Whole message
Resource Usage	Reserved for session	Shared among users	Shared, store-and-forward
Delay	Setup delay only	Transmission delay per packet	High delay (store-and-forward)
Efficiency	Low (idle time wasted)	High	Moderate
Use Case	Telephone network	Internet	Telegraph, email
Order of Delivery	In order	May vary	In order

6) Compare routing protocols RIP, OSPF, BGP

1. RIP (Routing Information Protocol)

Type:

→ Distance Vector Routing Protocol

Key Features:

- Uses **hop count** as the metric (max 15 hops).
- Periodically sends the **entire routing table** to neighbors.
- Simple and suitable for **small networks**.
- **Routing updates** every 30 seconds.

Advantages:

- Easy to configure and manage.
- Low resource usage.

Disadvantages:

- Slow convergence.
- Not scalable for large networks.
- Hop limit (15) restricts network size.

2. OSPF (Open Shortest Path First)

Type:

→ Link State Routing Protocol

Key Features:

- Uses **cost (based on bandwidth)** as the metric.
- Each router builds a **complete map of the network** using LSAs (Link State Advertisements).
- Supports **hierarchical design** using areas.
- **Fast convergence** and supports **VLSM/CIDR**.

Advantages:

- Efficient and scalable.
- Supports large and complex networks.
- Updates are only sent when topology changes (not periodic).

Disadvantages:

- More complex to configure.
- Higher memory and CPU usage.

3. BGP (Border Gateway Protocol)

Type:

→ Path Vector Routing Protocol

Key Features:

- Used for routing between **different autonomous systems (AS)** — i.e., **inter-domain routing**.
- Uses **path attributes** (AS path, policy, etc.) to choose the best route.
- Basis of **Internet routing**.
- Provides **policy-based routing** control.

Advantages:

- Highly scalable (used by ISPs and on the Internet).
- Allows control over routing policies between organizations.
- Very stable once routes are established.

Disadvantages:

- Very complex configuration.
- Slower convergence compared to OSPF.

4. Comparison Table

Feature	RIP	OSPF	BGP
Type	Distance Vector	Link State	Path Vector
Metric Used	Hop Count	Cost (Bandwidth)	Path Attributes
Max Hop Count	15	No limit	No limit
Convergence Speed	Slow	Fast	Moderate to Slow
Scalability	Small networks	Large enterprise networks	Very large (Internet)
Update Type	Periodic	On change	Incremental
Used For	Intra-domain	Intra-domain	Inter-domain
Protocol Number	UDP Port 520	IP Protocol 89	TCP Port 179
Complexity	Simple	Moderate	High
Example Use	Small LAN	Enterprise LAN/WAN	Internet backbone

7) Explain the concept of connection less and connection oriented protocol with example.

1. Connection-Oriented Protocol

Definition:

A **connection-oriented protocol** establishes a **dedicated connection** between sender and receiver before data transmission begins.

It ensures **reliable communication**, where data is delivered **in order** and **without errors**.

Key Features:

- Connection setup, data transfer, and termination phases.
- Acknowledgements are used to confirm receipt.
- Retransmission occurs if packets are lost.
- Provides **error checking**, **flow control**, and **reliability**.

Example:

- **TCP (Transmission Control Protocol)**
 - Used in web browsing (HTTP), email (SMTP), and file transfer (FTP).
 - Example: When you open a website, TCP establishes a connection between your device and the web server using a **three-way handshake**.

Analogy:

Like a **telephone call** — connection is established before conversation.

2. Connectionless Protocol

Definition:

A **connectionless protocol** sends data **without establishing a dedicated connection**.

Each data packet is treated **independently**, and delivery is **not guaranteed**.

Key Features:

- No connection setup or termination.
- Faster but **less reliable**.
- No acknowledgements or retransmissions.
- Suitable for applications where **speed is more important than reliability**.

Example:

- **UDP (User Datagram Protocol)**
 - Used in live streaming, online gaming, and voice/video calls.
 - Example: In a video call, if a few packets are lost, the conversation continues without delay.

Analogy:

Like sending **letters by post** — each is sent individually without confirming delivery.

3. Difference Table

Feature	Connection-Oriented Protocol (TCP)	Connectionless Protocol (UDP)	
Connection Setup	Required (3-way handshake)	Not required	
Reliability	Reliable (acknowledgements, retransmission)	Unreliable (no confirmation)	
Data Delivery Order	In sequence	May arrive out of order	
Speed	Slower	Faster	
Overhead	High	Low	
Example	TCP (HTTP, FTP, SMTP)	UDP (DNS, VoIP, Streaming)	
Analogy	Telephone call	Postal letters	

8) Give the different functions of the Network Layer.

Network Layer (Layer 3 of OSI Model)

Definition:

The **Network Layer** is responsible for **data delivery from source to destination across multiple networks**.

It deals with **logical addressing, routing, and path determination**.

Main Functions of the Network Layer

1. Logical Addressing:

- Assigns **unique IP addresses** to devices.
- Ensures each packet can be identified and delivered to the correct destination network.

2. Routing:

- Determines the **best path** for data to travel from source to destination.
- Uses **routing algorithms and protocols** (e.g., RIP, OSPF, BGP).

3. Packet Forwarding:

- Forwards data packets from one network to another based on routing tables.

4. Fragmentation and Reassembly:

- If a packet is too large for the underlying network, it is **divided (fragmented)**.
- At the destination, fragments are **reassembled** into the original packet.

5. Error Handling and Diagnostics:

- Detects and reports network-level errors (e.g., unreachable host).
- Uses tools like **ICMP (Internet Control Message Protocol)** for reporting.

6. Internetworking:

- Allows communication between **different types of networks** (LANs, WANs) by providing a universal addressing scheme.

7. Congestion Control (optional):

- Helps manage network traffic to **avoid congestion** and packet loss.

Summary Table

Function	Description
Logical Addressing	Assign IP addresses to identify devices
Routing	Selects best path for data
Packet Forwarding	Moves packets toward destination
Fragmentation & Reassembly	Breaks/rejoins packets for transmission
Error Handling	Reports errors via ICMP
Internetworking	Connects different networks
Congestion Control	Manages traffic flow to prevent overload

9) Explain in detail, any two network layer protocols.

1. Internet Protocol (IP)

Definition:

The **Internet Protocol (IP)** is the **primary protocol of the Network Layer** responsible for delivering packets from the source to the destination across interconnected networks.

It provides **logical addressing** and **routing** functions.

Functions of IP:

1. Logical Addressing:

- Every device on a network is assigned a unique **IP address** (e.g., 192.168.1.5).
- This helps identify the **source and destination** of each packet.

2. Packet Routing:

- IP determines the **best path** for data to travel using routing tables.
- Routers use the **destination IP address** to forward packets to the next hop.

3. Fragmentation and Reassembly:

- If a packet is too large for a network, IP divides it into **smaller fragments**.
- The destination host **reassembles** the fragments into the original message.

4. Encapsulation:

- IP encapsulates **Transport Layer segments (TCP/UDP)** into **IP packets** before sending them to the Data Link Layer.

5. Error Handling:

- Works with **ICMP (Internet Control Message Protocol)** to report transmission errors (like unreachable hosts).

Versions of IP:

Version	Full Form	Address Size	Example
IPv4	Internet Protocol version 4	32-bit	192.168.1.10
IPv6	Internet Protocol version 6	128-bit	2001:0db8::1

Example:

When a user visits a website:

Your computer's IP (source) → sends a request packet to the web server's IP (destination) using routers that forward the packet along the best path.

2. Internet Control Message Protocol (ICMP)

Definition:

The **Internet Control Message Protocol (ICMP)** is used by network devices (like routers) to **send error messages and operational information** about network communication.

Functions of ICMP:

1. Error Reporting:

- Reports problems like:
 - Destination unreachable
 - Time exceeded (packet TTL expired)
 - Source quench (network congestion)

2. Network Diagnostics:

- Used by tools like **ping** and **traceroute** to check connectivity and route paths.
- Example:
 - `ping` → Tests if a host is reachable.
 - `traceroute` → Shows the path packets take across routers.

3. Flow Control Assistance:

- Helps routers and hosts manage network congestion by notifying when data cannot be delivered.

ICMP Message Types:

Type	Message Name	Purpose
0	Echo Reply	Response to ping
3	Destination Unreachable	Host/network not reachable
8	Echo Request	Sent by ping to check connectivity
11	Time Exceeded	TTL expired before reaching destination

Example:

If you try to access a server that's offline, your router sends back an **ICMP "Destination Unreachable"** message to notify you that the server cannot be reached.

10) What is Routing? Explain the different routing protocols

(REFER QUESTION NO 6 FOR ROUTING PROTOCOLS)

Definition:

Routing is the process of **selecting the best path** for data to travel from the **source** to the **destination** across interconnected networks.

In simple terms — it's how routers decide **where to send packets next** so they reach their final destination.

Functions of Routing:

1. Path Determination:

Chooses the most efficient route based on metrics like hop count, bandwidth, delay, or cost.

2. Packet Forwarding:

Forwards data packets from one router to the next toward the destination.

3. Network Discovery:

Routers learn about other networks and build routing tables using protocols.

Types of Routing

Type	Description	Example
1. Static Routing	Routes are manually set by the administrator.	Small networks
2. Dynamic Routing	Routers automatically learn and update routes using routing protocols.	Large/complex networks
3. Default Routing	Used when no specific route is known; all unknown packets are sent to a default router.	Gateway to Internet

11) Write a short note on: Mobile IP.

Mobile IP

Definition:

Mobile IP is a **network layer protocol** that allows a device (called a *mobile node*) to move from one network to another **without changing its IP address**.

It enables **seamless Internet connectivity** even when the user moves across different networks.

Need for Mobile IP:

In traditional IP, when a device changes its network, its **IP address also changes**, breaking ongoing connections.

Mobile IP solves this problem by keeping the same home IP address regardless of location.

Key Components:

1. Mobile Node (MN):

The device that moves from one network to another (e.g., smartphone, laptop).

2. Home Agent (HA):

A router on the **home network** that maintains the mobile node's permanent IP address and forwards packets to it.

3. Foreign Agent (FA):

A router on the **visited (foreign) network** that helps the mobile node connect and communicate.

4. Care-of Address (CoA):

A temporary IP address assigned to the mobile node while it's on the foreign network.

Working of Mobile IP:

1. When the mobile node moves to a new network, it registers its **Care-of Address** with its **Home Agent**.
 2. The **Home Agent** intercepts packets sent to the mobile node's **home address** and **tunnels** them to the **Care-of Address**.
 3. The **Foreign Agent** receives the tunneled packets and delivers them to the mobile node.
 4. Replies from the mobile node are sent directly to the destination host.
-

Advantages:

- Maintains ongoing connections while moving across networks.
- Provides seamless Internet access for mobile devices.
- Supports mobility across different networks and subnets.

Applications:

- Mobile phones and laptops with wireless Internet.
- IoT and vehicular networks.
- Remote work and mobile computing.

12) Explain Link State Routing Algorithm with example

Link State Routing Algorithm

Definition:

Link State Routing (LSR) is a routing algorithm in which **each router has complete knowledge of the network topology** and independently calculates the **shortest path** to every other router using this information.

It is a **Link State Protocol**, used in protocols like **OSPF** and **IS-IS**.

Key Features:

1. Each router **discovers its neighbors** and their link costs.
2. Routers create **Link State Packets (LSPs)** containing information about neighbors and link costs.
3. LSPs are **flooded** to all routers in the network.
4. Each router builds a **complete network map** (topology database).
5. Routers use **Dijkstra's Shortest Path First (SPF) algorithm** to compute the shortest path to each destination.

Steps of the Algorithm:

1. **Discover Neighbors:** Each router identifies directly connected routers and link costs.
2. **Create Link State Packet (LSP):** Contains router ID, neighbors, and cost of each link.
3. **Flood LSPs:** LSPs are sent to all routers in the network so each router has the same information.
4. **Build Topology Table:** Each router constructs a network graph using all received LSPs.
5. **Compute Shortest Path:** Apply **Dijkstra's algorithm** to calculate the shortest path to all destinations.
6. **Update Routing Table:** Based on the shortest paths, routing table entries are created for packet forwarding.

Example:

Consider a network with 4 routers (A, B, C, D) and the following link costs:

Link	Cost
A – B	1
A – C	4
B – C	2
B – D	6
C – D	3

Step 1: Discover Neighbors

- Router A knows B (1) and C (4)
- Router B knows A (1), C (2), D (6)
- Router C knows A (4), B (2), D (3)
- Router D knows B (6), C (3)

Step 2: Create LSPs and Flood

- Each router sends its LSP to all other routers.
- After flooding, all routers have the same network topology.

Step 3: Compute Shortest Path (Dijkstra)

- From **Router A**:
 - Path to B = $A \rightarrow B = 1$
 - Path to C = $A \rightarrow B \rightarrow C = 1 + 2 = 3$ (shorter than direct $A \rightarrow C = 4$)
 - Path to D = $A \rightarrow B \rightarrow C \rightarrow D = 1 + 2 + 3 = 6$

Step 4: Routing Table for A

Destination	Next Hop	Cost
B	B	1
C	B	3
D	B	6

Advantages of Link State Routing:

- Fast convergence compared to Distance Vector Routing.
- Each router has complete knowledge of the network.
- Supports hierarchical networks (areas) and scalable designs.
- Prevents routing loops.

Disadvantages:

- Requires more memory to store network topology.
- Requires more CPU for Dijkstra calculations.
- High overhead for flooding LSPs in large networks.

✅ In short:

Link State Routing allows each router to have a **full map of the network**, computes shortest paths using **Dijkstra's algorithm**, and builds a **routing table** for efficient and loop-free packet delivery.

13) Explain Distance Vector Routing Algorithm with example.

Distance Vector Routing Algorithm

Definition:

Distance Vector Routing (DVR) is a routing algorithm in which each router maintains a **routing table** containing the **distance (cost) to reach each destination** and the **next hop (vector)**.

Routers **periodically share their routing tables with immediate neighbors** to update their knowledge of the network.

Key Protocols: RIP (Routing Information Protocol) uses DVR.

Key Features:

1. Each router knows only about its **directly connected neighbors** initially.
2. Routers exchange **distance vectors** (routing tables) with neighbors periodically.
3. Each router updates its routing table using the **Bellman-Ford equation**:

$$D_x(y) = \min\{c(x, v) + D_v(y)\}$$

Where:

- $D_x(y)$ = Distance from router X to destination Y
 - $c(x, v)$ = Cost to reach neighbor V
 - $D_v(y)$ = Distance from neighbor V to destination Y
4. Routers gradually learn the **shortest path** to all destinations.

Steps of the Algorithm:

1. Initialization:

- Each router sets the distance to itself = 0, and to all other routers = ∞ .

2. Exchange Information:

- Routers send their distance vector to all neighbors periodically.

3. Update Routing Table:

- Each router updates its table based on neighbors' vectors using the formula above.

4. Convergence:

- The process repeats until all routers have consistent shortest path distances.

Example:

Consider a network with 3 routers: A, B, C

Link	Cost
A – B	2
A – C	5
B – C	1

Step 1: Initialization

- Router A: Distance to A=0, B=2, C=5
- Router B: Distance to B=0, A=2, C=1
- Router C: Distance to C=0, A=5, B=1

Step 2: Exchange distance vectors

- A receives B's vector: B→C=1
- Update A→C via B: A→B→C = 2+1=3 (better than 5)
- Update A's routing table: A→C = 3 via B

Step 3: Updated Routing Tables

Router A:

Destination	Next Hop	Cost	
B	B	2	
C	B	3	

Router B:

Destination	Next Hop	Cost
A	A	2
C	C	1

Router C:

Destination	Next Hop	Cost
A	B	3
B	B	1

Step 4: Convergence

- After several updates, all routers have **shortest paths to every destination**.

Advantages of Distance Vector Routing:

- Simple and easy to implement.
- Requires less memory than Link State Routing.
- Suitable for small networks.

Disadvantages:

- Slow convergence, prone to **routing loops**.
- Uses **periodic updates** even when no changes occur.
- Less efficient for large networks.

✅ In short:

Distance Vector Routing relies on routers **sharing their distance tables with neighbors**.

Each router gradually computes the **shortest path to all destinations** using the **Bellman-Ford algorithm**.

UNIT 4

14) Explain TCP Handles Error control and Flow Control

TCP Handles Error Control and Flow Control

Transmission Control Protocol (TCP) is a **reliable, connection-oriented transport layer protocol**. It ensures **error-free, orderly, and congestion-free delivery** of data between sender and receiver. Two key mechanisms that help TCP achieve reliability are **Error Control** and **Flow Control**.

✖ 1. Error Control in TCP

Definition:

Error control ensures that data is **delivered correctly and without corruption**.

TCP uses several techniques to detect and correct errors during transmission.

Mechanisms used:

1. Checksum:

- Each TCP segment includes a **checksum field**.
- The sender calculates the checksum and adds it to the header.
- The receiver recalculates it — if it doesn't match, the segment is considered **corrupted** and is discarded.

2. Acknowledgments (ACKs):

- After receiving a segment correctly, the receiver sends an **ACK** back to the sender.
- This ensures reliable delivery.

3. Retransmission (ARQ - Automatic Repeat Request):

- If an ACK is not received within a specific **timeout period**, the sender retransmits the lost segment.

4. Sequence Numbers:

- Each segment is assigned a **sequence number** to keep track of data order.
- This allows the receiver to **detect missing or duplicate segments** and reorder them properly.

● Example:

If the sender sends segments 1, 2, and 3, and ACK for 2 is not received, TCP retransmits segment 2.

⚙️ 2. Flow Control in TCP

Definition:

Flow control ensures that the **sender does not overwhelm the receiver** with too much data at once.

Mechanism used:

◆ Sliding Window Protocol:

- TCP uses a **window-based flow control** mechanism.
- The receiver advertises a **window size** (number of bytes it can accept).
- The sender can send data only up to that window size without waiting for an ACK.
- When the receiver processes data and sends ACKs, it updates the window size.

● Example:

If the receiver's buffer size is **5000 bytes**, the sender will not send more than 5000 bytes until an acknowledgment is received.

✅ Summary Table:

Feature	Error Control	Flow Control	📄
Purpose	Detect and correct errors	Prevent receiver overload	
Mechanism	Checksum, ACK, retransmission	Sliding window protocol	
Reliability	Ensures correct delivery	Ensures smooth data rate	
Example	Retransmit lost segment	Limit data by window size	

In short:

- **Error Control** in TCP ensures data is received **accurately** using **checksum, ACKs, and retransmission**.
- **Flow Control** ensures data is sent at a **rate the receiver can handle**, using the **sliding window mechanism**.

Together, they make TCP a **reliable and efficient transport protocol**.

15) Explain Different elements of Transport Protocol

Different Elements of Transport Protocol

Definition:

A **Transport Protocol** is responsible for **reliable end-to-end communication** between processes running on different hosts.

It lies between the **network layer** and the **application layer** in the OSI model.

Examples: **TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)**.

The **main elements** of a transport protocol define how data is transmitted, controlled, and managed between sender and receiver.

⚙️ 1. Addressing

- The transport layer identifies communication endpoints using **port numbers**.
- Each process (application) is assigned a **unique port number** to distinguish multiple connections on the same host.
- Example:
 - HTTP → Port **80**
 - FTP → Port **21**
 - SMTP → Port **25**

📦 2. Multiplexing and Demultiplexing

- **Multiplexing:**
Allows multiple application processes to share the same transport connection.
Data from multiple processes is combined and sent over the network.
- **Demultiplexing:**
At the receiver's end, data is separated based on **port numbers** and delivered to the correct process.

✅ Example:

Your computer can browse the web (HTTP) and send emails (SMTP) at the same time through multiplexing.

🔄 3. Connection Establishment and Termination

- For connection-oriented protocols like TCP:
 - A **connection** must be established before data transfer using a **3-way handshake** (SYN, SYN-ACK, ACK).
 - After data transfer, the connection is **terminated gracefully** using a **4-step termination** process.

✅ Ensures reliable and synchronized communication between both ends.

4. Flow Control

- Prevents the **sender from overwhelming the receiver**.
 - Implemented using the **Sliding Window Protocol** in TCP.
 - The receiver advertises its buffer capacity (window size) so the sender sends only as much data as the receiver can handle.
-

5. Error Control

- Ensures **error-free data delivery** through mechanisms like:
 - **Checksum** (for error detection)
 - **Acknowledgment (ACK)**
 - **Retransmission of lost packets**
- TCP uses **sequence numbers** to maintain correct order of segments and detect missing or duplicate data.

6. Segmentation and Reassembly

- **Segmentation:**
Large messages from the application layer are divided into smaller segments by the transport layer for transmission.
 - **Reassembly:**
At the receiver's end, these segments are reassembled into the original message using sequence numbers.
-

7. Connection Control (Reliable vs Unreliable Service)

- TCP provides a **reliable, connection-oriented** service (guaranteed delivery).
- UDP provides an **unreliable, connectionless** service (faster but no guarantee).

✔ Summary Table

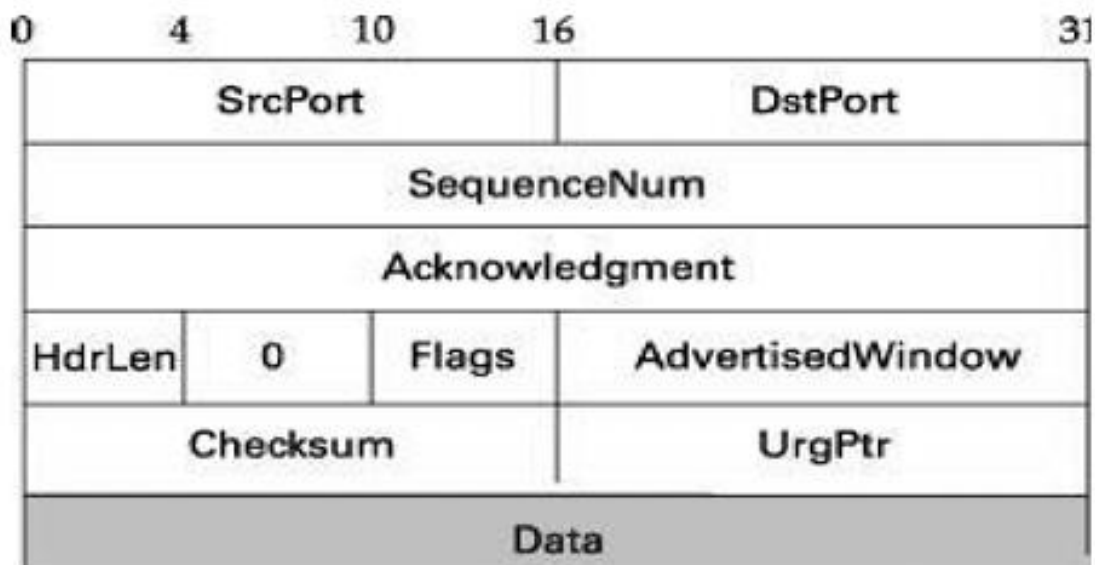


Element	Function
Addressing	Identifies sending/receiving processes using port numbers
Multiplexing/Demultiplexing	Allows multiple applications to share one network link
Connection Establishment/Termination	Controls start and end of communication
Flow Control	Regulates data rate between sender and receiver
Error Control	Detects and corrects errors in data transmission
Segmentation/Reassembly	Divides and reassembles data for transmission
Connection Control	Provides reliable or unreliable delivery depending on protocol

16) Draw and Explain TCP Segment Header.

TCP SEGMENT

- A packet in TCP is called a segment.
- Data unit exchanged between TCP peers are called **segments**.
- A TCP segment encapsulates the data received from the application layer.

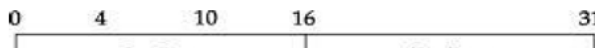


TCP PACKET FORMAT

Each TCP segment contains the header plus the data.

The segment consists of a header of 20 to 60 bytes, followed by data from the application program.

The header is 20 bytes if there are no options and up to 60 bytes if it contains options.



- **SrcPort and DstPort**—port number of source and destination process.
- **SequenceNum**—contains sequence number, i.e. first byte of data segment.
- **Acknowledgment**— byte number of segment, the receiver expects next.
- **HdrLen**—Length of TCP header as 4-byte words.
- **Flags**— contains *six* control bits known as flags.
 - o **URG** — segment contains urgent data.
 - o **ACK** — value of acknowledgment field is valid.
 - o **PUSH** — sender has invoked the push operation.
 - o **RESET** — receiver wants to abort the connection.
 - o **SYN** — synchronize sequence numbers during connection establishment.
 - o **FIN** — terminates the TCP connection.
- **Advertised Window**—defines receiver's window size and acts as flow control.
- **Checksum**—It is computed over TCP header, Data, and pseudo header containing IP fields(Length, SourceAddr & DestinationAddr).
- **UrgPtr** — used when the segment contains urgent data. It defines a value that must be added to the sequence number.
- **Options** - There can be up to 40 bytes of optional information in the TCP header

17) Discuss the Quality-of-Service Parameter in Computer Network.

Quality of Service (QoS) in Computer Networks

Definition:

Quality of Service (QoS) refers to the performance level of a network service.

It defines how efficiently and reliably data is transmitted from source to destination.

QoS ensures that important applications (like video calls, VoIP, streaming, etc.) get the required bandwidth, low delay, and minimal packet loss.

Need for QoS:

- Different applications have different requirements.
 - Example: Voice and video need low delay and jitter.
 - File transfer needs high reliability.
- QoS helps manage network resources to meet these varied demands.

Quality-of-Service Parameters

The main parameters that define QoS are:

1. Bandwidth (Throughput)

- It represents the data-carrying capacity of a network link (in bits per second).
- Higher bandwidth = more data transmitted per second.
- Example: A 100 Mbps connection provides higher throughput than 10 Mbps.

✅ Importance: Ensures sufficient capacity for high-data-rate applications (e.g., video streaming).

2. Delay (Latency)

- It is the time taken by a packet to travel from source to destination.
- Delay includes transmission delay, propagation delay, queuing delay, and processing delay.

✅ Importance:

Low delay is critical for real-time applications like VoIP and online gaming.

3. Jitter

- It is the **variation in packet delay** during transmission.
- Caused by **network congestion** or **variable queue lengths**.

✅ Importance:

High jitter causes **distorted audio or video** in real-time communication.

QoS mechanisms try to **minimize jitter**.

4. Packet Loss

- Occurs when packets are **dropped** due to congestion, errors, or buffer overflow.
- Measured as a **percentage** of lost packets over total sent packets.

✅ Importance:

Even small packet loss can degrade **voice, video, or file transfer quality**.

5. Reliability

- Refers to the **consistency and dependability** of a network connection.
- Depends on error rates, packet delivery ratio, and connection stability.

✅ Importance:

High reliability ensures **accurate and complete data transfer** (important for banking, emails, etc.).



6. Availability

- The **percentage of time** a network or service is operational and accessible.
- Usually measured as **uptime** (e.g., 99.9% availability).

✅ Importance:

Essential for continuous services like cloud computing and online transactions.

7. Security (Optional QoS Parameter)

- Protects data against **unauthorized access or modification** during transmission.

✅ Importance:

Ensures **confidentiality and integrity** of sensitive information.

Summary Table

Parameter	Description	Importance
Bandwidth	Data transfer capacity	Needed for high-speed apps
Delay	Time for data to reach destination	Crucial for real-time apps
Jitter	Variation in delay	Affects video/voice quality
Packet Loss	Dropped packets	Reduces data integrity
Reliability	Accuracy and consistency	Ensures dependable communication
Availability	Operational uptime	Keeps network accessible
Security	Data protection	Ensures confidentiality

In short:

Quality of Service (QoS) measures how well a network meets user expectations. It depends on parameters like **bandwidth**, **delay**, **jitter**, and **packet loss**, which together ensure **efficient**, **reliable**, and **real-time** communication.

18) Write short note on sockets and sockets programming

Sockets and Socket Programming

1. What is a Socket?

A **socket** is an **endpoint for communication** between two devices (processes) across a network. It allows applications to **send and receive data** just like reading and writing to a file.

A socket is defined by:

👉 (IP Address, Port Number) pair.

So, a socket uniquely identifies a process on a networked computer.

2. Types of Sockets:

1. Stream Sockets (TCP):

- Use **Transmission Control Protocol (TCP)**.
- Provide **reliable, connection-oriented** communication.
- Example: Web browsing, email, file transfer.

2. Datagram Sockets (UDP):

- Use **User Datagram Protocol (UDP)**.
- Provide **connectionless, fast but unreliable** communication.
- Example: Online gaming, video streaming.

3. What is Socket Programming?

Socket programming is the process of **writing programs that communicate over a network** using sockets.

It allows two systems (client and server) to **exchange data**.

4. Basic Steps in Socket Programming:

Server Side:

1. **Create a socket** → `socket()`
2. **Bind** it to an IP address and port → `bind()`
3. **Listen** for incoming connections → `listen()`
4. **Accept** a client connection → `accept()`
5. **Send/Receive** data → `send()` / `recv()`
6. **Close** the connection → `close()`

Client Side:

1. **Create a socket** → `socket()`
2. **Connect** to the server → `connect()`
3. **Send/Receive** data → `send()` / `recv()`
4. **Close** the connection → `close()`

6. Applications:

- Web browsers and servers
- Chat and messaging applications
- File transfer systems
- Online games and streaming services

In short:

A **socket** acts as a **communication bridge** between networked applications.

Socket programming enables **client-server communication** over TCP/IP networks, forming the foundation of most **Internet-based applications**.

19) Explain RTP protocol in detail.

RTP (Real-Time Transport Protocol)

1. Definition:

The **Real-Time Transport Protocol (RTP)** is a **network transport protocol** designed for **real-time applications** such as:

- Audio and video streaming
- Video conferencing
- Voice over IP (VoIP)

It provides **end-to-end delivery services** for data with real-time characteristics such as **timing, sequencing, and synchronization**.

2. Purpose of RTP:

Traditional transport protocols like **TCP** are reliable but introduce delay (due to retransmission), which is **not suitable for real-time data**.

Hence, **RTP** is used over **UDP** (User Datagram Protocol) to ensure **low latency** transmission of multimedia data, even if a few packets are lost.

3. Characteristics of RTP:

- Works on **Transport Layer**, usually **over UDP**.
- Provides **timing information** and **sequence numbers** for synchronization.
- Does **not guarantee delivery** — but supports **monitoring and control** through RTCP.
- Used for **unicast** and **multicast** communication.

4. Components of RTP:

(a) RTP Data Protocol

- Carries the actual **media data (audio, video, etc.)**.
- Adds **RTP header** containing:
 - **Sequence Number**: To detect packet loss and maintain correct order.
 - **Timestamp**: Used for synchronization and jitter calculation.
 - **Payload Type**: Identifies the type of data (e.g., audio codec like MP3, video codec like H.264).
 - **Source Identifier (SSRC)**: Uniquely identifies the source of the stream.

(b) RTCP (RTP Control Protocol)

- Works alongside RTP.
- Used for **quality monitoring and feedback**.
- RTCP packets provide:
 - **Sender reports**: Transmission statistics (e.g., packet count, delay, jitter).
 - **Receiver reports**: Quality feedback to the sender.
 - **Synchronization information** between multiple streams (e.g., audio and video).

5. Working of RTP:

1. Sender:

- Captures media (audio/video).
- Encodes it into frames.
- Adds RTP headers (sequence number, timestamp).
- Sends packets via **UDP**.

2. Receiver:

- Receives RTP packets.
- Uses **sequence numbers** to reorder packets.
- Uses **timestamps** for playback timing.
- RTCP reports help adjust transmission quality if needed.

7. Advantages of RTP:

- Provides **timing and sequencing** for real-time data.
- Works efficiently over **UDP** with **low latency**.
- Supports **multimedia synchronization** (audio + video).
- Provides **quality monitoring** via RTCP.
- Widely supported in VoIP, video conferencing, and streaming apps.

8. Applications of RTP:

- **VoIP (Voice over IP)**
- **Video Conferencing** (Zoom, Teams, Skype)
- **Online Streaming** (YouTube Live, Twitch)
- **Multimedia Broadcasts**
- **IPTV Services**

✅ In short:

RTP (Real-Time Transport Protocol) is a **transport layer protocol** that provides **timing, sequencing, and synchronization** for real-time multimedia communication.

It works with **RTCP** for **monitoring and control**, ensuring **smooth playback** and **low latency** in applications like **VoIP, video conferencing, and streaming**.

20) Compare TCP and UDP Transport layer protocols.

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
1. Connection Type	Connection-oriented (connection established before data transfer).	Connectionless (no connection setup needed).
2. Reliability	Reliable – ensures data delivery using acknowledgments and retransmissions.	Unreliable – no acknowledgment or retransmission.
3. Error Control	Performs error detection, acknowledgment, and retransmission of lost packets.	Performs only error detection (checksum), no correction.
4. Flow Control	Uses flow control (Sliding Window, Congestion Control).	No flow control mechanism.
5. Sequencing	Data is transmitted in sequence ; receiver reassembles data in correct order.	No sequencing – packets may arrive out of order.
6. Header Size	Larger (20–60 bytes) – includes many control fields.	Smaller (8 bytes) – minimal overhead.
7. Speed	Slower due to reliability mechanisms.	Faster as there is no error checking or connection setup.
8. Use Case	Suitable for applications where accuracy is critical.	Suitable for real-time applications where speed is more important than reliability.
9. Examples	HTTP, HTTPS, FTP, SMTP, Telnet.	DNS, DHCP, VoIP, Online Games, Video Streaming.
10. Acknowledgment	Each segment is acknowledged by the receiver.	No acknowledgment is sent.

3. Diagram (Conceptual Difference)

pgsql

 Copy code

TCP (Connection-Oriented)

Client ----- SYN -----> Server

Client <----- SYN-ACK ----- Server

Client ----- ACK -----> Server

(Connection established before data transfer)

UDP (Connectionless)

Client ----- DATA -----> Server

(No connection setup, just sends data)

4. Summary

- TCP is used when accuracy, order, and reliability are required.
- UDP is used when speed and efficiency are more important than reliability.

21) What is Congestion? Explain the different causes of congestion.

What is Congestion?

Definition:

Congestion in a computer network occurs when the **demand for network resources exceeds the available capacity**, causing **packet loss, delay, or reduced performance**.

In simple terms, congestion happens when **too many packets** are present in the network, and routers or switches **cannot handle the load efficiently**.

Example:

When many users try to access the Internet or download files simultaneously, routers may become overloaded, causing **delays or dropped packets** — this is called **network congestion**.

Causes of Congestion

There are several factors that lead to congestion in a network:

1. High Traffic Load

- When **too many packets** are sent into the network at once, the routers' **buffer gets full**.
- As a result, some packets must be **dropped or delayed**.

📌 *Example:* Many users streaming videos simultaneously on the same network.

2. Insufficient Bandwidth

- If the **link capacity** (bandwidth) is less than the traffic load, packets accumulate in queues.
- Leads to **bottlenecks** and **increased delay**.

📌 *Example:* Using a 10 Mbps link for traffic demanding 50 Mbps.

3. Slow Network Devices

- Routers and switches with **limited processing power** cannot handle high-speed data efficiently.
- Causes **packet queuing and delay**.

📌 *Example:* An older router being used in a high-speed LAN environment.

4. Long Queue Size / Buffer Overflow

- Each router has a **buffer** to store packets temporarily.
- If packets arrive faster than they are processed, buffers **overflow**, leading to **packet loss**.

✂ *Example:* Congested router drops packets when its buffer is full.

5. Bursty Traffic (Sudden Increase)

- Sudden **spikes in data transmission** cause temporary overload.
- Such bursty traffic can cause routers to become congested even if average traffic is normal.

✂ *Example:* Many users sending large emails at the same time.

6. Poor Congestion Control Mechanisms

- Some protocols (like UDP) **don't use congestion control**, leading to excessive data transmission.
 - TCP has mechanisms like **slow start** and **congestion avoidance**, but if not implemented correctly, congestion occurs.
-

7. Retransmissions due to Packet Loss

- When packets are lost, **senders retransmit** them, adding **more load** to the already congested network.
- This creates a **vicious cycle**, worsening congestion.

In Summary:

Cause	Explanation
High traffic load	Too many packets at once
Low bandwidth	Link cannot handle data rate
Slow devices	Routers/switches process slowly
Buffer overflow	Queue becomes full
Bursty traffic	Sudden surge in data
Poor congestion control	No proper flow regulation
Retransmissions	Repeated packet sending increases load

✔ In short:

Congestion occurs when **network resources are overused**, leading to **delays, packet loss, and degraded performance**.
Causes include **high traffic, limited bandwidth, slow routers, buffer overflow, and poor congestion control**.

22) Explain Congestion Control

Congestion Control in Computer Networks

Definition:

Congestion Control is the process of **managing network traffic** to prevent or reduce congestion. It ensures that **network resources are used efficiently**, **packet loss is minimized**, and **throughput is maximized**.

In simple words, congestion control **prevents the network from getting overloaded**.

Objectives of Congestion Control:

1. Avoid excessive **packet loss**.
2. Minimize **network delay**.
3. Ensure **fair resource allocation** among all users.
4. Maintain **high throughput and efficiency**.

Principles of Congestion Control

- Regulate the **rate at which packets are injected** into the network.
- Detect congestion **early** using feedback or network signals.
- Adjust **sending rate dynamically** to match network capacity.

Techniques of Congestion Control

1. Open-Loop Congestion Control (Preventive)

- **Prevents congestion** before it occurs.
- Uses mechanisms like:
 - **Admission control:** Limit the number of users or connections.
 - **Traffic shaping / policing:** Regulate data entering the network.
 - **Scheduling and prioritization:** Give priority to critical data.

📌 *Example:* Limiting video streaming rate to avoid network overload.

2. Closed-Loop Congestion Control (Reactive)

- **Detects congestion** and **reacts** to reduce it.
- Methods include:
 - **Backpressure:** Congested nodes inform upstream nodes to slow down.
 - **Choke packets:** Special packets sent to sources to reduce transmission rate.
 - **Packet dropping:** Router drops some packets to signal congestion to the sender.

📌 *Example:* TCP reduces the sending rate when packet loss is detected.

Advantages of Congestion Control:

- Prevents **network collapse** due to overload.
 - Reduces **packet loss and delays**.
 - Ensures **fairness** among multiple users.
 - Maintains **high throughput and efficiency**.
-

In short:

Congestion control is a set of techniques used to **prevent or reduce network congestion**.

TCP implements it using **Slow Start, Congestion Avoidance, and Fast Recovery**, ensuring **efficient, fair, and reliable communication** even under high network load.

23) Explain SCTP Transport Layer Protocols

SCTP (Stream Control Transmission Protocol)

1. Definition:

SCTP is a **transport layer protocol** that provides **reliable, connection-oriented communication** like TCP, but with **additional features** such as multi-streaming and multi-homing.

It is defined in **RFC 4960** and primarily used for **telecommunications signaling (e.g., SS7 over IP)** and **highly reliable applications**.

2. Key Features of SCTP

1. Reliable Data Transfer:

- Like TCP, SCTP ensures **reliable, in-order delivery** of messages using acknowledgments and retransmissions.

2. Message-Oriented:

- SCTP is **message-based** (preserves message boundaries) rather than **byte-stream based** like TCP.

3. Multi-Streaming:

- Allows multiple **independent streams** within a single SCTP connection.
- **Prevents head-of-line blocking** (one lost packet does not block other streams).

4. Multi-Homing Support:

- An SCTP endpoint can have **multiple IP addresses**.
- Provides **fault tolerance**: if one path fails, traffic switches to another without dropping the connection.

5. Four-Way Handshake:

- Connection setup uses **four-way handshake**, providing **protection against SYN flooding attacks** (unlike TCP's 3-way handshake).

6. Congestion Control and Flow Control:

- Similar to TCP, SCTP implements **congestion avoidance** and **sliding window flow control**.

7. Path MTU Discovery:

- Determines the **maximum transmission unit** on a path to avoid fragmentation.

5. Applications of SCTP

- Signaling in **telecommunication networks** (SS7 over IP)
 - **High-availability servers** requiring fault tolerance
 - **WebRTC and streaming applications** for message delivery
 - **Distributed systems** needing multi-streaming and reliable messaging
-

6. In Short

- SCTP is a **reliable, connection-oriented transport protocol** like TCP but **message-oriented**.
- Its **multi-streaming and multi-homing features** make it suitable for **telecommunication and high-reliability applications**.
- SCTP combines the **reliability of TCP** with the **efficiency and flexibility of message-oriented protocols**.

24) What are the techniques to improve Quality of Service (QoS)?

Techniques to Improve Quality of Service (QoS)

Definition of QoS:

Quality of Service (QoS) is the **ability of a network to provide better service** to selected network traffic. It ensures that critical applications (like video conferencing, VoIP, and streaming) get **required bandwidth, low delay, and minimal packet loss**.

Techniques to Improve QoS

QoS can be improved using **various mechanisms at different layers** of the network:

1. Traffic Shaping

- Controls the **flow of outgoing traffic** to ensure smooth delivery.
- Limits the rate at which packets are sent into the network to **prevent congestion**.
- Common techniques: **Leaky Bucket** and **Token Bucket algorithms**.

📌 *Example:* Regulating video streaming to prevent sudden spikes.

2. Traffic Policing

- **Monitors network traffic** and enforces rules.
 - Excess traffic can be **dropped or marked** for lower priority.
 - Ensures that users **do not exceed their allocated bandwidth**.
-

3. Packet Scheduling and Prioritization

- Routers schedule packets based on **priority levels**.
 - High-priority traffic (e.g., voice or video) is transmitted first.
 - Common algorithms:
 - **Priority Queuing (PQ)**
 - **Weighted Fair Queuing (WFQ)**
 - **Round Robin (RR)**
-

4. Resource Reservation

- **Reserves network resources** like bandwidth for specific applications or flows.
- Example: **RSVP (Resource Reservation Protocol)** reserves bandwidth for multimedia sessions.

✂ Ensures that **critical applications always have sufficient resources**.

5. Congestion Management and Control

- Prevents network congestion which can degrade QoS.
 - Techniques include:
 - **Active Queue Management (AQM)** like RED (Random Early Detection)
 - **TCP Congestion Control** to adjust sending rates dynamically.
-

6. Error Control and Reliability Enhancement

- Ensures **data integrity** and reduces packet loss.
 - Techniques:
 - **Forward Error Correction (FEC)**
 - **Retransmissions for lost packets**
 - **Checksum and CRC for error detection**
-

7. Differentiated Services (DiffServ)

- Classifies traffic into **different service levels**.
- Packets are marked with a **DSCP (Differentiated Services Code Point)** to indicate priority.
- Enables **scalable QoS for large networks**.

8. Admission Control

- Limits the number of users or sessions admitted into the network.
- Ensures that new sessions do **not degrade the QoS** of existing sessions.

Summary Table

Technique	Purpose	
Traffic Shaping	Smooth out bursts, prevent congestion	
Traffic Policing	Enforce bandwidth limits and rules	
Packet Scheduling	Prioritize important traffic	
Resource Reservation	Guarantee bandwidth for critical flows	
Congestion Control	Prevent overload and packet loss	
Error Control	Improve reliability and reduce retransmissions	
Differentiated Services	Provide multiple service levels	
Admission Control	Maintain QoS by limiting new sessions	

UNIT 5

25) Write Short Note on SNMP

SNMP (Simple Network Management Protocol)

Definition:

SNMP stands for **Simple Network Management Protocol**.

It is an **Application Layer protocol** used for **monitoring, managing, and controlling network devices** such as routers, switches, servers, printers, and computers in an IP network.

Purpose:

SNMP enables **network administrators** to:

- Monitor network **performance**
- Detect and **troubleshoot problems**
- Manage network **configurations and resources** remotely

Components of SNMP:

Component	Description
1. Managed Device	The network device being monitored (e.g., router, switch, server).
2. Agent	Software running on a managed device that collects data and reports to the manager.
3. Network Management System (NMS)	Central system that monitors and controls devices via agents.

Working of SNMP:

1. **SNMP Manager (NMS)** sends **requests** to agents on devices.
2. **SNMP Agent** collects device information (like CPU usage, bandwidth, errors).
3. Agent responds with the requested data using **SNMP messages**.
4. If any fault occurs, the agent sends a **trap message** to the manager automatically.

SNMP Operations (Messages):

- **GET**: Retrieve information from an agent.
- **SET**: Change a configuration on a device.
- **GET-NEXT**: Retrieve the next item in a list.
- **TRAP**: Notification from agent to manager about an event/fault.

Management Information Base (MIB):

- A **database of network objects** (like interfaces, devices, statistics).
 - Each object is identified by an **Object Identifier (OID)**.
 - SNMP uses the MIB to **locate and retrieve** information from devices.
-

Versions of SNMP:

1. **SNMPv1**: Basic version, limited security.
 2. **SNMPv2**: Added performance improvements and bulk data retrieval.
 3. **SNMPv3**: Added **authentication and encryption** for security.
-

Advantages:

- Simple and easy to implement.
 - Supports centralized monitoring.
 - Works on both small and large networks.
-

In short:

SNMP is a protocol that allows **network administrators to monitor, manage, and control network devices** efficiently using agents and a management system.

It helps in **fault detection, configuration, and performance monitoring**.

26) Explain in Brief about Telnet

Telnet (Terminal Network Protocol)

Definition:

Telnet stands for **Terminal Network**.

It is an **application layer protocol** that allows a user to **remotely access another computer or network device** over a **TCP/IP network** (such as the Internet).

In simple words, Telnet lets a user **log in to a remote system** and work as if they were physically present at that machine.

Port Number:

- **Default Port: 23 (TCP)**

Working of Telnet:

1. The **Telnet client** connects to a **Telnet server** using TCP port 23.
2. Once connected, the user can **enter commands** on the client system.
3. These commands are **executed on the remote host**, and the output is sent back to the client.
4. It provides a **command-line interface (CLI)** for remote system control.

Modes of Operation:

- **Client Mode:** The system initiating the connection.
- **Server Mode:** The remote system being accessed.

Functions of Telnet:

- Provides **remote login** capability.
- Allows **execution of commands** on a remote computer.
- Enables **network device management** (e.g., routers, switches).
- Useful for **troubleshooting and testing** network connectivity.

Advantages:

- Simple to use and implement.
 - Allows full control of remote systems.
 - Useful for configuring and monitoring network devices.
-

Disadvantages:

- **No encryption** — all data (including passwords) is sent in **plain text**.
 - Vulnerable to **security attacks** like eavesdropping.
 - Replaced by **SSH (Secure Shell)** for secure remote access.
-

In short:

Telnet is a **text-based protocol** that enables **remote login and command execution** on networked systems.

It operates on **TCP port 23** but is now largely replaced by **SSH** due to its **lack of security**.

27) Explain Simple mail Transfer Protocol

Simple Mail Transfer Protocol (SMTP)

Definition:

SMTP stands for **Simple Mail Transfer Protocol**.

It is an **application layer protocol** used for **sending and forwarding email messages** between mail servers over the Internet.

In simple terms, SMTP is the **standard protocol used to send emails** from one computer (client) to another (server).

Port Numbers:

- **Port 25** – Default port for SMTP (server-to-server communication).
- **Port 587** – Used for sending mail from client to mail server (submission).
- **Port 465** – Used for **secure SMTP** (with SSL/TLS encryption).

Working of SMTP:

SMTP follows a **client-server model** and uses **TCP** as the transport layer protocol (for reliable delivery).

Steps in Email Transmission:

1. Mail User Agent (MUA):

The email is composed using an email client (e.g., Outlook, Gmail).

2. Mail Submission:

The MUA sends the message to the **Mail Transfer Agent (MTA)** using SMTP.

3. Mail Transfer:

The sender's MTA forwards the email through one or more intermediate servers to the recipient's MTA.

4. Mail Delivery:

The recipient's MTA stores the message, and the user retrieves it using **POP3** or **IMAP**.

Example of SMTP Commands:

Command	Meaning / Function
HELO	Identify the sender's domain to the mail server.
MAIL FROM:	Specify the sender's email address.
RCPT TO:	Specify the recipient's email address.
DATA	Start sending the body of the message.
QUIT	End the session.

Features of SMTP:

- Uses **TCP for reliable delivery**.
- Supports **relaying** (forwarding mail across multiple servers).
- Supports **plain text communication** (can be secured using SSL/TLS).
- Handles only **sending** of emails (not retrieval).

Limitations:

- Cannot **retrieve** or **download** emails (uses POP3/IMAP for that).
- Transmits messages in **plain text** (insecure without encryption).
- Cannot send **multimedia attachments** directly (uses MIME for that).

In Short:

SMTP is an **application layer protocol** that handles the **sending and forwarding of emails** between mail servers using **TCP ports 25, 465, or 587**.

It works with **POP3** or **IMAP** for retrieving messages and is the **backbone of email communication** on the Internet.

28) Write short note on DNS

Domain Name System (DNS)

Definition:

DNS (Domain Name System) is an **application layer protocol** that translates **human-readable domain names** (like `www.google.com`) into **IP addresses** (like `142.250.190.78`) required by computers to locate each other on the Internet.

In simple terms, DNS works as the **"phonebook of the Internet."**

Need for DNS:

- Humans remember **names**, not **numbers**.
- Computers use **IP addresses** for communication.
- DNS bridges this gap by **mapping domain names to IP addresses** automatically.

Working of DNS:

When a user types `www.example.com` in a browser:

1. **DNS Resolver (Client side):** Sends a query to find the IP address.
2. **Root DNS Server:** Directs the query to the **Top-Level Domain (TLD)** server (like `.com`, `.org`, etc.).
3. **TLD Server:** Points to the **Authoritative DNS Server** for that domain.
4. **Authoritative DNS Server:** Returns the **IP address** of the website.
5. **Browser:** Uses this IP to contact the web server and load the website.

DNS Components:

Component	Description	
DNS Resolver	The client that initiates the DNS query.	
Root Server	The top-level DNS server that knows all TLD servers.	
TLD Server	Handles domains under a specific extension (.com, .org, .in).	
Authoritative Server	Contains the actual IP address of the domain.	

Types of DNS Records:

Record Type	Purpose
A	Maps a domain name to an IPv4 address.
AAAA	Maps a domain name to an IPv6 address.
CNAME	Alias or canonical name record.
MX	Mail exchange record (for emails).
NS	Identifies the authoritative name servers.

Advantages of DNS:

- Easy to remember domain names.
- Efficient and fast domain resolution using caching.
- Provides load balancing and redundancy.
- Supports scalability for global networks.

In Short:

DNS is a **distributed naming system** that converts **domain names into IP addresses**, enabling users to access websites easily without remembering numeric IPs.

29) Explain POP Protocol.

POP (Post Office Protocol)

Definition:

POP stands for **Post Office Protocol**.

It is an **application layer protocol** used by **email clients** to **retrieve emails** from a **mail server**.

The latest version used today is **POP3 (Post Office Protocol Version 3)**.

In simple words, POP allows users to **download emails from a mail server** to their local computer and read them **offline**.

Port Numbers:

- **Port 110** – Default (non-secure)
- **Port 995** – Secure (POP3S, uses SSL/TLS)

Working of POP:

1. The **email client** (like Outlook, Thunderbird, etc.) connects to the **mail server** using POP3.
2. The client **downloads** all the emails to the local system.
3. By default, emails are **deleted from the server** after downloading.
4. The user can then **read emails offline**.

Steps in POP Communication:

1. **Connect:** Client establishes a TCP connection with the mail server.
2. **Authenticate:** User logs in using username and password.
3. **Retrieve:** Client downloads email messages.
4. **Delete (optional):** Server deletes emails after transfer.
5. **Quit:** Connection is closed.

Commands Used in POP3:

Command	Description
USER	Sends the username
PASS	Sends the password
LIST	Lists all email messages
RETR	Retrieves a specific message
DELE	Deletes a message
QUIT	Ends the session

Advantages:

- Simple and easy to implement.
- Allows **offline access** to emails.
- Reduces server storage load since emails are stored locally.

Disadvantages:

- Emails are **removed from the server** after downloading (by default).
- Cannot **sync across multiple devices**.
- No access to sent or draft folders on the server.

Difference between POP3 and IMAP:

POP3

Downloads emails to local device.

Works offline.

No synchronization between devices.

IMAP

Keeps emails on the server.

Requires internet connection.

Synchronizes across all devices.

In Short:

POP3 is a **simple email retrieval protocol** that lets users **download emails** from a server and **read them offline**.

It operates on **TCP port 110** (or **995** for secure POP3S).

30) Explain various FTP commands

FTP (File Transfer Protocol) Commands

Definition:

FTP (File Transfer Protocol) is an **application layer protocol** used to **transfer files** between a **client** and a **server** over a **TCP/IP network**.

It operates on **port 21** for command control and **port 20** for data transfer.

Working:

1. The **FTP client** connects to the **FTP server** using a username and password.
2. The user can **upload, download, rename, delete, or list files** using FTP commands.
3. FTP supports two modes:
 - **Active Mode**
 - **Passive Mode**

Common FTP Commands and Their Functions

Command	Meaning / Function
USER	Sends the username to log in to the FTP server.
PASS	Sends the password for authentication.
LIST	Displays a list of files and directories on the server.
PWD	Prints the current working directory on the server.
CWD	Changes the working directory on the server.
CDUP	Moves one directory up in the server directory structure.
RETR	Downloads a file from the server to the client.
STOR	Uploads a file from the client to the server.
DELE	Deletes a file from the server.
RNFR	Specifies the file to rename (from) .
RNTO	Specifies the new name (to) for the file.
MKD	Creates a new directory on the server.
RMD	Removes (deletes) a directory from the server.

TYPE	Sets the file transfer mode (e.g., ASCII or Binary).
QUIT	Ends the FTP session and disconnects.
HELP	Displays a list of available FTP commands.
PORT	Specifies the client's port number for data connection (active mode).
PASV	Requests the server to open a passive data port for transfer.

Example FTP Session:

shell

 Copy code

```
ftp> open ftp.example.com
Connected to ftp.example.com
Name: user123
Password: *****
ftp> ls
ftp> cd uploads
ftp> put report.txt
ftp> get data.zip
ftp> quit
```

In Short:

FTP commands are used to **log in**, **navigate directories**, **upload/download files**, and **manage data** between a client and a remote server.
It uses **port 21 for control** and **port 20 for data transfer**.

31) Explain the working of Dynamic Host Configuration Protocol (DHCP).

Dynamic Host Configuration Protocol (DHCP)

Definition:

DHCP stands for **Dynamic Host Configuration Protocol**.

It is a **network management protocol** used to **automatically assign IP addresses** and other network configuration parameters (like subnet mask, gateway, DNS) to devices (clients) on a network.

In simple words, DHCP **automatically provides IP configuration** to devices, so users don't have to set IP addresses manually.

Purpose of DHCP:

- To **dynamically allocate IP addresses** to devices.
- To **prevent IP conflicts** (duplicate IPs).
- To **simplify network administration**.

Port Numbers:

- **UDP Port 67** → used by the **DHCP Server**
- **UDP Port 68** → used by the **DHCP Client**

Working of DHCP:

When a device (client) joins a network, the DHCP process goes through **four main steps**, often remembered as **DORA**:

Step	Message Type	Description
1. DHCP Discover	Broadcast	The client broadcasts a DHCP Discover message to find available DHCP servers on the network.
2. DHCP Offer	Broadcast/Unicast	A DHCP server responds with a DHCP Offer message, offering an available IP address and configuration details.
3. DHCP Request	Broadcast	The client sends a DHCP Request message to accept the offered IP address.
4. DHCP Acknowledgment (ACK)	Unicast	The server sends a DHCP ACK message confirming that the IP address is assigned to the client.

Information Provided by DHCP:

- IP Address
- Subnet Mask
- Default Gateway
- DNS Server Address
- Lease Duration (how long IP is valid)

Advantages of DHCP:

- **Automatic IP assignment** (no manual setup).
- **Prevents IP conflicts.**
- **Centralized management** of IP addresses.
- **Efficient use** of available IPs.

Disadvantages:

- If the DHCP server fails, new devices **can't get IP addresses.**
- Not suitable for devices needing **static IPs** (like servers).

In Short:

DHCP is an **automatic IP configuration protocol** that uses the **DORA process** (Discover, Offer, Request, Acknowledge) to assign IP addresses dynamically to devices on a network.

32) Define HTTP. Explain persistent and Non-Persistent HTTP

HTTP (Hypertext Transfer Protocol)

Definition:

HTTP stands for **Hypertext Transfer Protocol**.

It is an **application layer protocol** used for **transmitting hypermedia documents** (like HTML, images, videos) between a **client** (web browser) and a **web server** over the Internet.

In simple terms, HTTP is the **foundation of data communication on the World Wide Web**, enabling users to access web pages using URLs (like `https://www.google.com`).

Default Port:

- **Port 80** → HTTP
- **Port 443** → HTTPS (Secure HTTP with SSL/TLS)

Basic Working of HTTP:

1. The **client (browser)** sends an **HTTP request** to the server.
2. The **server** processes the request and sends an **HTTP response** (usually containing the requested web page).
3. The connection may **close or remain open**, depending on the HTTP type (persistent or non-persistent).

Types of HTTP Connections

1. Non-Persistent HTTP

- A **new TCP connection** is created **for each request/response pair**.
- After the server sends the response, the **connection is closed immediately**.
- For a webpage containing multiple objects (HTML, CSS, images), **multiple connections** are required.

Example:

If a webpage has 3 images, the browser opens **3 separate TCP connections** to fetch them one by one.

Advantages:

- Simple implementation.
- Each connection is short-lived.

Disadvantages:

- **High overhead** due to repeated TCP connection setup and teardown.
- **Slower page loading** for multiple objects.

2. Persistent HTTP (Keep-Alive)

- A **single TCP connection** is maintained for **multiple request/response transactions** between client and server.
- The connection **remains open** until the client or server decides to close it.

Example:

If a webpage has 3 images, all 3 can be fetched over **one TCP connection**.

Advantages:

- **Reduced latency** (faster page load).
- **Less overhead** (no repeated connection setup).
- **Efficient use** of network resources.

Disadvantages:

- Server resources are **held longer** (connection remains open).
- Slightly **more complex** to manage idle connections.

Difference between Persistent and Non-Persistent HTTP

Feature	Non-Persistent HTTP	Persistent HTTP
Connections per object	One new connection per object	Single connection for multiple objects
Connection closing	After each response	After all responses or timeout
Speed	Slower	Faster
Overhead	High	Low
Efficiency	Less efficient	More efficient

In Short:

HTTP is the **protocol for web communication**, using **request and response messages**.

- **"Non-persistent HTTP opens a new connection for each request."**
- **"Persistent HTTP keeps a single connection alive for multiple requests, improving speed and efficiency."**

33) Explain HTTP request and reply message format.

HTTP Message Format

HTTP (Hypertext Transfer Protocol) uses two types of messages for communication between the client and server:

1. HTTP Request Message → Sent by the client to the server.
2. HTTP Response (Reply) Message → Sent by the server back to the client.

1. HTTP Request Message Format

Structure:

```
php-template
```

 Copy code

```
<request-line>
<headers>
<blank line>
<message body>
```

a) Request Line

The first line specifies what the client wants to do.

It has three parts:

```
mathematica
```

 Copy code

```
Method   URL   HTTP-Version
```

Example:

```
pgsql
```

 Copy code

```
GET /index.html HTTP/1.1
```

Part	Description
Method	Indicates the action to perform. Common methods: <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code> , <code>HEAD</code> .
URL	The path of the resource on the server (e.g., <code>/index.html</code>).
HTTP Version	Version of the HTTP protocol (e.g., <code>HTTP/1.1</code>).

b) Header Fields

Contain metadata about the request. Each header line has a key–value pair format.

Examples:

```
makefile
```

 Copy code

```
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Content-Type: application/json
```

Header Type	Purpose
General Header	Common information (e.g., <code>Date</code> , <code>Connection</code>).
Request Header	Info about client or resource (e.g., <code>User-Agent</code>).
Entity Header	Info about the message body (e.g., <code>Content-Length</code>).

c) Blank Line

A single empty line separates headers from the message body.

d) Message Body

Contains data sent by the client (used mostly in `POST` or `PUT` requests).

Example:

```
ini
```

 Copy code

```
username=karan&password=1234
```

✓ Example of a Complete HTTP Request

```
makefile
```

 Copy code

```
GET /home.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome/141.0
Accept-Language: en-US
```



2. HTTP Response (Reply) Message Format

Structure:

php-template

 Copy code

```
<status-line>
<headers>
<blank line>
<message body>
```

a) Status Line

The first line shows the response status.

css

 Copy code

```
HTTP-Version  Status-Code  Reason-Phrase
```

Example:

HTTP/1.1 200 OK

 Copy code

Part	Description
HTTP Version	The version used by the server.
Status Code	Numeric code showing result (e.g., 200, 404, 500).
Reason Phrase	Text message describing the status.

Common Status Codes:

Code	Meaning
200	OK – Request successful
301	Moved Permanently
400	Bad Request
404	Not Found
500	Internal Server Error

b) Header Fields

Provide information about the server and response.

Examples:

yaml

 Copy code

Date: Sun, 26 Oct 2025 10:00:00 GMT

Server: Apache/2.4.57

Content-Type: text/html

Content-Length: 2048

c) Blank Line

Separates header section from the actual data.

d) Message Body

Contains the actual content or resource requested by the client (like HTML page, image, etc.)

Example:

css

 Copy code

```
<html>
  <body>
    <h1>Welcome to Example.com!</h1>
  </body>
</html>
```

✓ Example of a Complete HTTP Response

php-template

 Copy code

```
HTTP/1.1 200 OK
Date: Sun, 26 Oct 2025 10:00:00 GMT
Server: Apache/2.4.57
Content-Type: text/html
Content-Length: 1256
```

```
<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

In Short:

Part	HTTP Request	HTTP Response
First Line	Request Line (<code>GET /index.html HTTP/1.1</code>)	Status Line (<code>HTTP/1.1 200 OK</code>)
Headers	Describe client, resource, data	Describe server, content type
Body	Sent with POST/PUT requests	Contains returned web content

Summary:

HTTP messages are plain text and structured for easy communication.

- "Request message: Client → Server (what is needed)"
- "Response message: Server → Client (what is delivered)"

34) Explain DNS frame format

DNS (Domain Name System) Frame Format

Definition:

The **DNS message (frame) format** defines how **queries** and **responses** are structured and exchanged between a **DNS client (resolver)** and a **DNS server**. Both **DNS query** and **response** messages use the **same format**, with some fields interpreted differently.

DNS Message Structure

Every DNS message consists of **five sections**:

pgsqlCopy code

```
+-----+
| 1. Header      |
+-----+
| 2. Question    |
+-----+
| 3. Answer      |
+-----+
| 4. Authority   |
+-----+
| 5. Additional  |
+-----+
```

1. Header Section (12 bytes)

The **header** appears in **both queries and responses** and contains general information about the message.

Field	Size (bits)	Description
Identification (ID)	16	A unique number assigned by the client. It helps match responses to queries.
Flags	16	Contains control bits and status flags (explained below).
QDCOUNT	16	Number of entries in the Question section.
ANCOUNT	16	Number of entries in the Answer section.
NSCOUNT	16	Number of entries in the Authority section.
ARCOUNT	16	Number of entries in the Additional section.

2. Question Section

Contains information about the **domain name being queried**.

Field	Description
QNAME	The domain name to be resolved (e.g., <code>www.example.com</code>).
QTYPE	Type of record requested (e.g., <code>A</code> , <code>MX</code> , <code>NS</code>).
QCLASS	Class of the query (usually <code>IN</code> for Internet).

3. Answer Section

Contains **resource records (RRs)** that answer the query.

Field	Description
NAME	Domain name that was queried.
TYPE	Type of record (e.g., <code>A</code> , <code>AAAA</code> , <code>MX</code>).
CLASS	Class (usually <code>IN</code>).
TTL	Time-to-Live — how long the record can be cached.
RDLENGTH	Length of RDATA field.
RDATA	Actual data (e.g., IP address for <code>A</code> record).

4. Authority Section

Gives information about the **authoritative name servers** for the domain.

Field	Description
Same structure as the Answer section	Lists authoritative DNS servers.

5. Additional Section

Contains **extra information** that may help in resolving the query (like IP addresses of name servers).

Field	Description
Same structure as the Answer section	Provides supplementary data.

Example: DNS Query Frame

java

 Copy code

Header:

```
ID = 0x1234
QR = 0 (Query)
Opcode = 0 (Standard)
QDCOUNT = 1
ANCOUNT = 0
NSCOUNT = 0
ARCOUNT = 0
```

Question:

```
QNAME = www.google.com
QTYPE = A
QCLASS = IN
```

Example: DNS Response Frame

vbnet

 Copy code

Header:

```
ID = 0x1234
QR = 1 (Response)
RCODE = 0 (No error)
QDCOUNT = 1
ANCOUNT = 1
```

Answer:

```
NAME = www.google.com
TYPE = A
CLASS = IN
TTL = 300
RDLENGTH = 4
RDATA = 142.250.190.78
```

In Short:

The **DNS frame format** consists of **Header**, **Question**, **Answer**, **Authority**, and **Additional** sections. It defines how DNS queries and responses are structured for **domain-to-IP translation**.

35) Explain FTP? Can we specify file transfer file in a Web page? Explain with the help of suitable example

FTP (File Transfer Protocol)

Definition:

FTP stands for File Transfer Protocol. It is a standard Internet protocol used to transfer files between a client and a server over a TCP/IP network (like the Internet).

It operates at the Application Layer of the OSI model and uses TCP ports 20 and 21:

- Port 21: Control connection (for commands and responses)
- Port 20: Data connection (for actual file transfer)

Working of FTP:

1. Connection Establishment:

The FTP client connects to the FTP server using TCP. A control connection is established on port 21.

2. Authentication:

The user provides a username and password to log into the FTP server.

3. Command and Data Channels:

- Control Channel: For sending commands like `LIST`, `STOR`, `RETR`, etc.
- Data Channel: For transferring files and directory listings.

4. File Transfer Modes:

- Active Mode: Server initiates data connection to the client.
- Passive Mode: Client initiates data connection to the server (used when firewalls block incoming connections).

Common FTP Commands:

Command	Description	
<code>USER</code>	Specifies username	
<code>PASS</code>	Specifies password	
<code>LIST</code>	Lists files in the current directory	
<code>RETR</code>	Downloads a file from server	
<code>STOR</code>	Uploads a file to server	
<code>CWD</code>	Changes working directory	
<code>PWD</code>	Shows current directory	
<code>QUIT</code>	Terminates the session	

Can We Specify File Transfer in a Web Page?

✅ Yes, we can specify a file transfer in a web page using the FTP URL format.

Example:

html

 Copy code

```
<a href="ftp://username:password@ftp.example.com/file.txt">Download File</a>
```

- ftp:// → protocol
- username:password@ → login credentials
- ftp.example.com → FTP server address
- /file.txt → file path on the server

When the user clicks the link, the browser initiates an FTP connection to the specified server and starts the file transfer.

Summary:

- FTP is used for uploading/downloading files between client and server.
- Uses TCP ports 20 (data) and 21 (control).
- Supports both **active** and **passive** modes.
- FTP links can be used directly in web pages for easy file access.

Example Web Link:

👉 `<a href="ftp://user:1234@ftp.myserver.com/report.pdf">Download Report</a>`